

API SECURITY CHECKLIST

Authentication

Authentication ensures that your users are who they say they are. Hackers that exploit authentication vulnerabilities can impersonate other users and access sensitive data.

- Drop Basic Authentication

Basic Authentication is the simplest form of HTTP authentication. With each request, users submit their credentials as plain and potentially unencrypted HTTP fields. Instead, use a more secure method such as [JWT](#) or [OAuth](#).

- Don't ship a home grown solution

Never try to implement your own authentication, token generation, or password storage methods. Depending on your application's language or framework, chances are there are existing solutions with proven security. Review the language or framework documentation to learn how to implement these solutions.

- Implement Max Retry and Jail safety mechanisms

Attackers will try to authenticate using a variety of credential combinations. Setting a maximum number of retries blocks users who fail too many authentication attempts in a certain amount of time. Users who exceed the number of max retries are placed in a "jail" which prevents further login attempts from their IP address until a certain amount of time passes.

- Encrypt Everything

Always encrypt data before transmission and at rest. Intercepting and reading plain HTTP is trivial for an attacker located anywhere between you and your users. Encryption makes it exponentially harder for credentials and other important information to be compromised.

Access

Attackers don't need to be authenticated in order to cause havoc.

- Limit Requests

One of the most common attacks on the Internet is a Denial of Service (DoS) attack, which involves sending a large number of requests to a server. The server tries to respond to each request and eventually runs out of resources. Rate limit requests to mitigate DoS attacks by throttling or blocking IP addresses and work with vendors that are able to block DoS attacks before they can even reach your servers.

- Force Encryption

Using unencrypted HTTP makes your users vulnerable to Man-In-The-Middle (MITM) attacks, which allows a hacker or third party to intercept sensitive data like usernames and passwords. Secure HTTP (HTTPS) encrypts data between clients and servers, preventing bad actors from reading this data.

Input

Just because users can log into your API doesn't mean they can be trusted. Failing to validate user input is the cause of some of the web's most debilitating vulnerabilities including Cross-Site Scripting (XSS) and SQL injections.

- Enforce HTTP Methods

Each of your API's endpoints should have a list of valid HTTP methods such as GET, POST, PUT, and DELETE. These methods should correlate to the action the user is attempting to perform (for example, GET should always return a resource, and DELETE should always delete a resource). Any operations that don't match those methods should return [405 Method Not Allowed](#). This prevents users from accidentally (or intentionally) performing the wrong action by using the wrong method.

- Perform Content Negotiation

When sharing data between the client and server, validate the type of content being sent. For example, if you expect the client to send JSON, only accept requests where the Content-Type header is set to application/json. If the content type isn't expected or supported, respond with [406 Not Acceptable](#).

- Validate User-Submitted Content

Malformed user input is the cause of some of the most common vulnerabilities on the web, including:

- **SQL Injection:** A user submits a malicious database query as input. If the application passes input directly to a database, the database will run the query.
- **Remote Code Execution:** A user submits a command as input. If the application passes input directly to a shell or external application, the server will run the command.
- **Cross-Site Scripting (XSS):** A user submits JavaScript as input. If a browser renders the field containing this JavaScript, the browser will execute the code.

You can mitigate these attacks by scrubbing user input of HTML tags, JavaScript tags, and SQL statements before processing it on the server.

Components with Vulnerabilities

Remove unused dependencies, unnecessary features, components, files, and documentation. Continuously check the versions of your dependencies for known security flaws.

- Check for trusted sources

When picking new dependencies only add code from official sources over secure links. Signed packages are ideal and reduce the chance of including a modified, malicious component into your application.

Data Processing

Scrubbing input won't always prevent you from attacks. Specially crafted payloads can still execute code on the server or even trigger a DoS.

- Protect Sensitive Endpoints

Make sure that all endpoints with access to sensitive data require authentication. This prevents unauthenticated users from accessing secure areas of the application and perform actions as anonymous users.

- Avoid Using Auto-Incrementing IDs

Auto-incrementing IDs make it trivial for attackers to guess the URL of resources they may not have access to. Instead, use universally unique identifiers (UUID) to identify resources.

- Process Data in the Background

Processing large amounts of data can prevent your API from responding in a timely manner. Instead of forcing the client to wait, consider processing the data asynchronously.

- Turn Debug Mode Off

While it may seem obvious, make sure your application is set to production mode before deployment. Running a debug API in production could result in performance issues, unintended operations such as test endpoints and backdoors, and expose data sensitive to your organization or development team.

Logging & Monitoring

Ensure all login, access control failures, and server-side input validation failures can be logged with sufficient user context to identify suspicious or malicious accounts, and held for sufficient time to allow delayed forensic analysis. Logs that are generated should be in a format that can be easily consumed by a centralized log management solution.